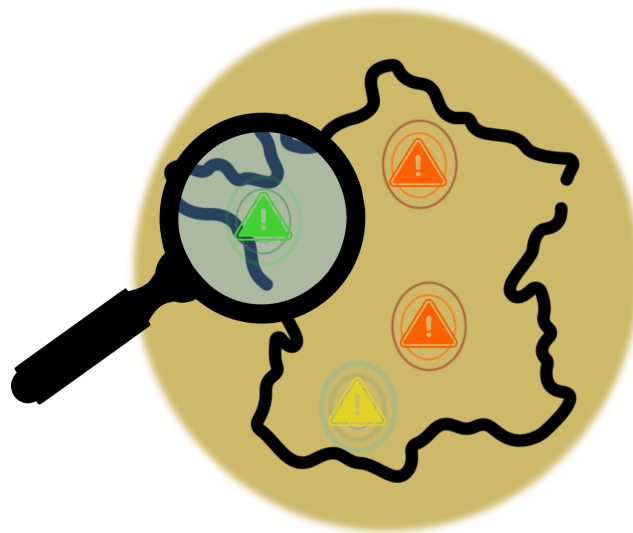


Projet Webmapping : Cartographie du danger



GeoWarning

Sommaire:

Introduction :	3
Méthode de développement	3
- Technologies utilisées.....	3
- Travail collaboratif et connectivité.....	4
- Répartition des tâches.....	5
- Temporalité.....	5
Le stockage des données avec PostgreSQL	5
- Le contenu de la base de donnée.....	5
- La mise à jour automatique de la base de données.....	8
L'envoi et la réception de données avec Geoserver	8
- La publication de couches.....	8
- L'hébergement des fichiers .html.....	9
Interface Homme-Machine	10
- Maquette de l'application.....	10
- Outils techniques.....	11
- Fonctionnalités en projet.....	11
Limites, Discussion et Piste d'amélioration	11
- Limites actuelle de l'application.....	11
- Limites dans le cadre d'un déploiement de l'application.....	12
Conclusion	13
Bibliographie :	14
Annexes	15

Introduction :

Dans le cadre de l'Unité d'Enseignement Webmapping, nous avons appris à créer et à manipuler une architecture trois-tiers. Des cours nous ont été dispensés par Stéphane Debard, Nicolas Moyroux et Jean-François Girres autour des trois éléments composant cette architecture (base de données, serveur géographique et interface homme-machine). A l'issue de ces enseignements la consigne nous a été donné de reprendre ces éléments pour :

- effectuer des recherches depuis une page web
- interroger une base de données
- renvoyer et afficher les résultats dans la page web

Dans ce cadre, nous avons créé GeoWarning un outil capable d'informer l'utilisateur des possibles dangers naturels qu'il encourt. Pour cela, notre outil s'appuie sur différentes sources de données :

- Les zones inondables
- Les zones de débroussaillage obligatoire (qui peuvent être associé à un risque accrue d'incendie)
- La qualité de l'air
- La vigilance Météo France en cours dans le département de l'utilisateur
- Les précipitations prévues et leur intensité autour de l'utilisateur

L'objectif de ce projet est que ce soit l'une de ces données ou bien le croisement de plusieurs sources qui donne lieu à une alerte pour l'utilisateur. Cette alerte sera caractérisée en fonction des types de danger détectés par l'application. Également, un autre objectif pour nous est de rendre cet outil portable, accessible partout où une connexion internet est disponible. Nous souhaitons également rendre l'outil disponible sur smartphone afin de simplifier son accès dans une situation potentiellement dangereuse pour l'utilisateur. Nous verrons toutefois que des limitations se posent sur ces deux derniers points.

Dans ce dossier, nous allons revenir sur les choix et les étapes par lesquelles nous sommes passés pour créer GeoWarning.

Méthode de développement

- Technologies utilisées

Pour créer notre outil nous avons dû faire des choix en matière de technologie que ce soit pour notre base de données, notre serveur ou notre interface homme-machine. Dans cette partie, nous allons revenir sur ces choix, en les expliquant et en faisant ressortir les avantages de ces choix et leurs limites.

Pour notre base de données, nous avons choisi PostgreSQL et son extension spatiale PostGIS, qui nous paraissait être le choix le plus approprié pour créer une base de données compte tenu des enseignements que nous avons suivis au cours du M1 sur cette technologie de base de données. Dans notre projet, son rôle est de stocker et de transformer les données. Pour la visualisation et la gestion de cette base de données, nous avons choisi DBeaver sur les conseils de M. Moiroux à la place de Pgadmin.

Côté serveur, nous avons choisi le serveur cartographique Geoserver. En effet, au cours des enseignements de M. Debard, son utilisation nous est apparue accessible. L'outil en lui-même nous a semblé complet et en mesure de s'intégrer à notre projet. Geoserver héberge également les fichiers .html permettant l'interface homme-machine.

Enfin, côté client, nous avons choisi Leaflet pour l'affichage des couches et les interactions avec Geoserver, comme appris dans les cours de M. Girres.

- Travail collaboratif et connectivité

Pour effectuer ce travail, nous avons choisi de mettre en place des technologies de travail collaboratif. Nous avons utilisé GitHub, pour le partage de fichiers et le versionnage du code développé.

Une autre technologie utilisée est Tailscale. C'est un logiciel qui permet l'interconnexion entre appareils. Ce logiciel nous permet de définir un ordinateur comme étant celui qui contient le serveur et la base de données et un deuxième ordinateur comme étant le client, qui veut accéder à l'application. Cette technologie nous permet d'accéder au serveur (et à son contenu hébergé) tant qu'une connexion internet est disponible. La disponibilité sur smartphone est également permise par ce logiciel. Cette organisation nous est également utile pour le développement, et nous permet de travailler chacun depuis son propre ordinateur. Le principe de cette application est de créer un tunnel sécurisé entre l'ordinateur (son port local) serveur et les périphériques qui souhaitent s'y connecter. Cela permet de simuler une architecture web classique (d'autant plus que les périphériques se connectent à partir d'une URL http classique) mais qui doivent être au préalable ajoutés sur l'application Tailscale pour des raisons de sécurité.

- Répartition des tâches

Tailscale nous a permis de nous répartir les tâches de manière plus efficiente, selon les points forts des membres du groupe. Ainsi, nous avons adopté la répartition suivante:

Samuel : Interface Homme-Machine, fonctionnalités, écriture du dossier

Maxime : Base de données, configuration et mise en place du serveur, écriture du dossier.

- Temporalité

D'un point de vue pratique, nous avons commencé par développer la partie stockage et gestion de la données, comprenant la base de données incluant la gestion des données en temps réel et le Geoserver. Puis, dans un second temps, nous avons créé la partie concernant les fonctionnalités et l'interface homme-machine de l'application. Le dossier et le logo ont également été créés dans cette seconde partie. La temporalité de notre projet s'étend de début décembre jusqu'à fin janvier.

Le stockage des données avec PostgreSQL

- Le contenu de la base de donnée

Comme évoqué dans l'introduction, notre base de données est composée de 5 sources de données, à savoir:

- Les zones inondables
- Les zones de débroussaillage obligatoire (qui peuvent être associées à un risque accru d'incendie)
- La qualité de l'air
- La vigilance Météo-France en cours dans le départements de l'utilisateur
- Les précipitations prévues et leur intensité autour de l'utilisateur

Les couches des communes, EPCI et départements sont également présentes afin de créer des vues et de transformer les données précédentes.

Parmi ces données, certaines sont figées comme les zones inondables, issues de Géorisques ou les zones de débroussaillage obligatoire, de l'IGN. Tandis que d'autres sont variables et dépendent d'API fournies par des fournisseurs tiers. C'est le cas de la qualité de l'air fournie par ATMO, de la vigilance et des précipitations fournies par Météo France.

Une chose importante est que nous avons choisi de ne pas faire entrer la pluie directement dans la base de données PostgreSQL pour l'affichage sur la carte, il

nous semblait compliqué de faire entrer une image raster dans une base de données. Nous avons préféré stocker cette image dans un fichier et indiquer le chemin de ce dernier à notre serveur. Toutefois, pour effectuer des calculs, nous utilisons une version polygonisée du raster de pluie.

Les tables de notre base de données sont organisées de cette manière :

Diagramme UML de la base de données de GeoWarning

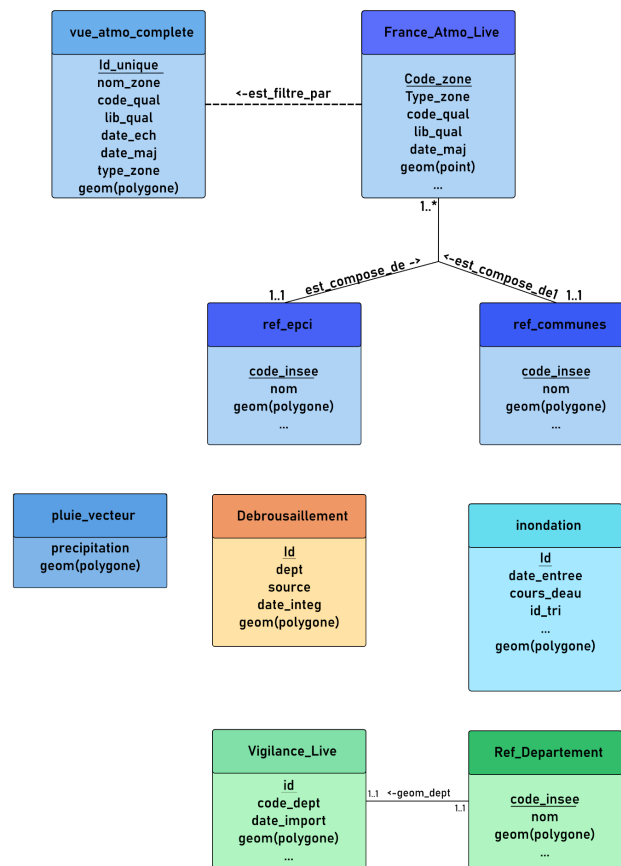


Figure 1: Schéma UML de la base de données

On remarque que la totalité des couches possède un attribut qui référence sa géométrie. Dans l'application, ne sont utilisées que des géométries de type polygone.

Nous allons maintenant détailler les différentes couches de la base de données en indiquant notamment leurs sources précises et leur rôle au sein de l'application GeoWarning et les choix que nous avons faits quant à leur représentation au sein de l'application. :

Premièrement, les zones inondables, appelées inondation dans la base de données. Cette donnée est issue de GéoRisques et plus spécifiquement du produit appelé "Zonages inondation - Rapportage 2020". Nous avons sélectionné les aléas de forte

probabilité (qui se produisent tous les 10 à 30 ans). Dans ce produit, nous avons sélectionné 3 couches que nous avons considérées comme essentielles :

- les débordements de cours d'eau (n_inondable_01_01for_s)
- le ruissellement (n_inondable_02_01for_s)
- les submersions marines (n_inondable_03_01for_s)

Nous ne traitons pas dans cette application les événements de plus faible probabilité. Cela aurait posé des contraintes de stockage et de temps de traitement, car nous tenons à proposer une application fonctionnelle sur l'ensemble de la France Métropolitaine. Pour l'affichage de cette couche, nous avons choisi de la représenter en bleu afin de rendre son interprétation aisée. Nous avons pour cela utilisé un style, créé grâce à QGIS. Nous voulons avertir l'utilisateur lorsqu'il se trouve dans une de ces zones. Une alerte plus visuelle pourra également être créée si un utilisateur se trouve dans cette zone et que de la pluie est prévue sur sa position (en lien avec les prévisions de pluie de Météo France)

Deuxièmement, nous utilisons la qualité de l'air, mise à jour de manière automatique (voir la section suivante) d'Atmo. Cette qualité est récupérée sous la forme de points, et contient un indice en 6 classes (de bon à extrêmement mauvais). Chez Atmo, ces données concernent les communes, excepté pour les régions de Bretagne et Occitanie où ces données renseignent des EPCI. Nous avons donc créé un script SQL qui se charge de cette transformation lors de l'arrivée de nouvelles données. Pour l'affichage de ces données, nous avons utilisé un style créé avec l'aide de QGIS et qui reprend les couleurs officielles définies par ATMO. L'idée est également d'envoyer une alerte à l'utilisateur lorsqu'il se trouve dans une zone où la qualité de l'air n'est pas suffisamment bonne.

Ensuite, nous avons choisi de faire figurer dans notre base de données les zones de débroussaillage, qui représentent pour nous les zones susceptibles d'être touchées par des incendies. Nous utilisons des données de Géorisques et plus précisément un produit nommé "Obligations Légales de Débroussaillage". Nous l'avons représentée en rouge et nous nous servons de cette donnée pour informer et alerter les utilisateurs de leur présence dans une zone incendie. Un croisement avec les alertes Météo France pour Canicule pourra être fait pour afficher une alerte plus prononcée.

La vigilance Météo France est une autre donnée présente dans notre base de données. Elle est également mise à jour de manière automatique, quotidiennement. Elle est représentée par le code couleur officiel de Météo France et nous nous servons de cette donnée pour afficher des alertes selon le niveau d'alerte émis par Météo France.

Enfin, la dernière couche présente dans notre application est la pluie. Comme décrit plus haut, cette couche n'est pas intégrée dans notre base de données PostgreSQL pour la visualisation. Cependant, pour la création d'alerte, le serveur va interroger

une version vectorisée de cette couche de pluie. Cela nous permet d'avoir une efficacité accrue sur l'affichage et des calculs simplifiés pour la création d'alertes. Concernant l'affichage de cette couche, nous avons choisi un dégradé de bleu.

- La mise à jour automatique de la base de données

Plusieurs éléments permettent la mise à jour automatique des tables vigilance (Cf. Annexe 1), pluie (Cf. Annexe 2) et qualité de l'air (Cf. Annexe 3) de notre base de données, à savoir :

- Un script Python
- Un petit script batch
- Le planificateur de tâche Windows

Pour chacune de ces tables, le planificateur de tâches déclenche (selon l'horaire ou le critère défini) l'exécution du petit script batch qui utilise les librairies de QGIS pour lancer le script en Python. Ce dernier a pour rôle d'aller télécharger les nouveaux fichiers à l'aide de l'API (grâce aux librairies requests et subprocess), puis dans un second temps, d'aller vider puis remplir la base de données avec ce nouveau fichier.

Ce système nous permet de nous passer d'un logiciel ETL (pour Extract Transform and Load) et permet d'être automatiquement lancé depuis le système de notre ordinateur ayant le rôle de serveur. Toutefois, il requiert une connexion internet et que les serveurs des fournisseurs tiers soient accessibles et opérationnels. Également, si l'ordinateur serveur n'est pas allumé, les données ne sont pas mises à jour. L'intelligence artificielle nous a aidé à développer ces scripts de mise à jour des données.

L'envoi et la réception de données avec Geoserver

- La publication de couches

L'ensemble de nos couches finales sont publiées dans 3 systèmes de projection. Nous avons dû utiliser Lambert 93 pour les couches de départements, de communes et des EPCI. De fait, les couches de qualité de l'air et de vigilance sont dans ce même système de coordonnées géographiques. Mais d'autres couches comme celles des pluies sont dans d'autres systèmes de coordonnées (WGS 84) comme le point GPS envoyé par l'interface. Le rôle du Geoserver est donc d'orchestrer tous ces systèmes de coordonnées et de reprojeter à la volée les couches selon la requête passée pour obtenir un résultat homogène et cohérent (pas de WGS 84 pour le calcul de surface par exemple).

L'autre intérêt de notre serveur géographique est de permettre la mise en cache des couches statiques. En effet, afin d'optimiser l'affichage de nos couches, nous avons choisi de mettre en cache les couches des zones d'inondation et de zones de débroussaillage. Cette mise en cache est effectuée entre les zooms 4 et 13, pour trouver un compromis entre espace de stockage et rapidité.

Le rôle de notre serveur géographique est l'émission de flux WMS (.png) et WFS(geojson). Cette émission se fera selon l'emprise de la carte affichée sur l'écran de l'utilisateur. Une exception à cette règle est la couche vue_atmo_complete relative à la qualité de l'air qui sera requêtée selon la commune ou l'EPCI dans lequel se trouve l'utilisateur. L'objectif est de faire des économies de calcul et de ne pas envoyer toute la couche à l'utilisateur.

Pour terminer cette partie nous avons fait un schéma général de l'architecture de GeoWarning.

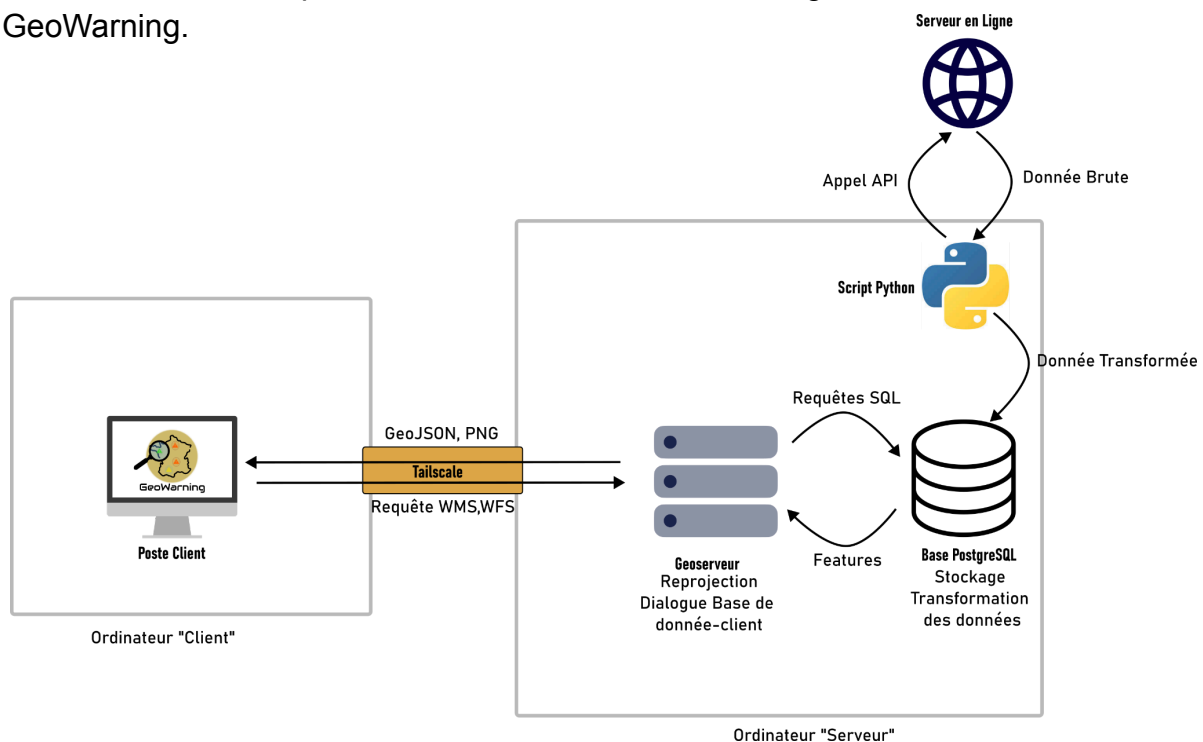


Figure 2: Schéma de fonctionnement de GeoWarning

- L'hébergement des fichiers .html

Un autre de nos objectifs est de faire héberger notre application par le Geoserver, les fichiers .html seront placés à l'intérieur des dossiers du serveur et seront disponibles uniquement à l'aide du lien http (sous réserve que l'appareil qui souhaite se connecter soit autorisé par le logiciel Tailscale).

Interface Homme-Machine

- Maquette de l'application

Ci-dessous la maquette de notre application:

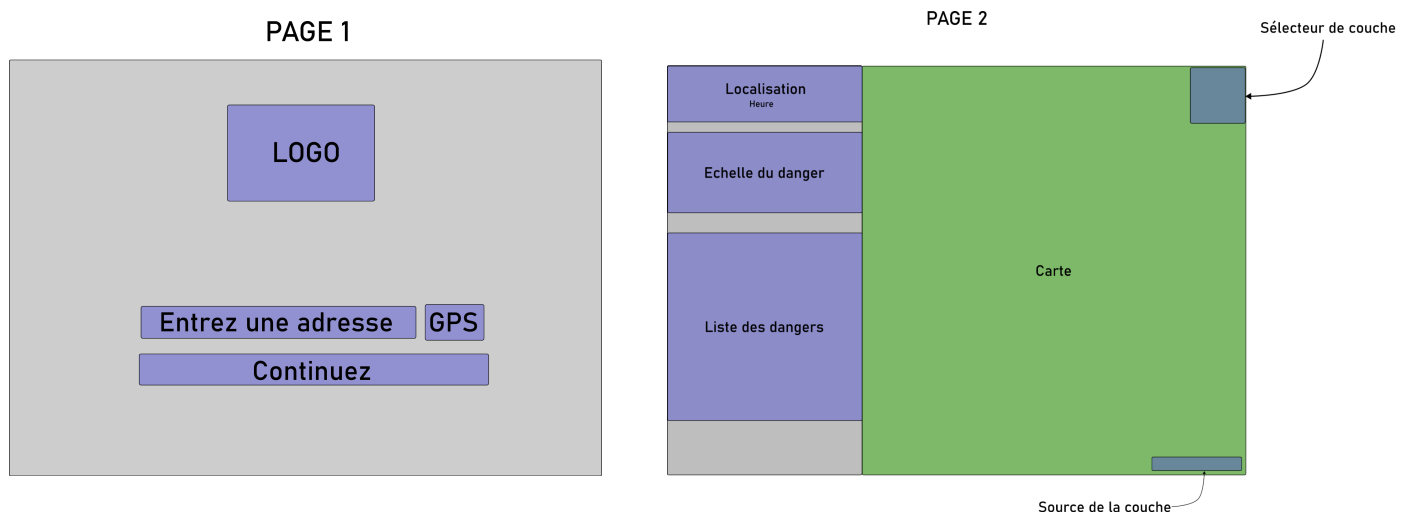


Figure 3 : Maquette de l'application GeoWarning

Notre application est donc composée de deux pages HTML. La première page sert à déterminer la position de l'utilisateur, en utilisant son adresse ou la géolocalisation de l'appareil. Une fois le bouton "Continuez" pressé, la deuxième page s'ouvre et affiche une carte avec la position de l'utilisateur. Sur la gauche, une liste indique les dangers potentiels qu'il encourt et une échelle de danger indique le niveau de risque.

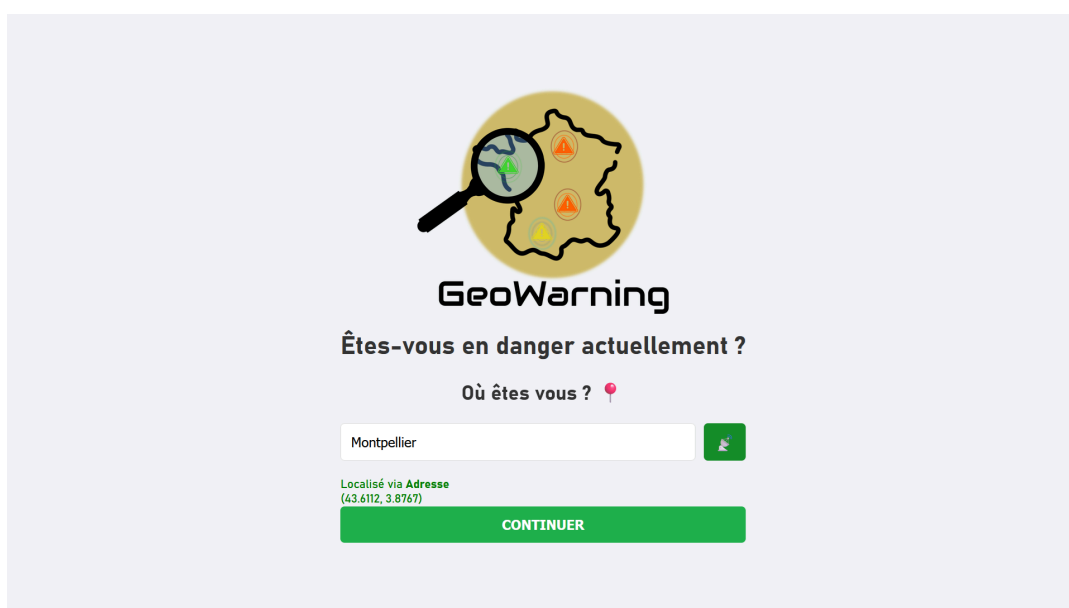


Figure 4: Première page de l'interface homme-machine

- Outils techniques

Pour réaliser cette interface homme-machine, nous utilisons Leaflet. Ainsi qu'un fond de carte OpenStreetMap nommé Carto Positron. Les langages utilisés sont:

- HTML pour la structure de la page
- CSS pour la partie esthétique de l'interface
- JavaScript pour l'interactivité avec le serveur

- Fonctionnalités en projet

Concernant GeoWarning, nous avons des fonctionnalités en projet d'ici la date de soutenance. Premièrement, nous souhaitons créer des alertes qui soient caractérisées, et que l'importance de l'alerte suive la grandeur du risque encouru. Exemple, si l'utilisateur se trouve dans un département en vigilance rouge il reçoit une alerte plus importante que s'il se trouve dans une vigilance jaune ou orange. On peut également prolonger un peu plus loin ce concept et par exemple afficher des alertes plus importantes si l'utilisateur se trouve dans une zone inondable lorsqu'il pleut plus de 10 mm par exemple.

Deuxièmement, nous avons l'ambition de faire héberger l'application par le Geoserver. L'objectif est qu'une fois que le client est connecté à Tailscale, il ait juste besoin de l'URL pour accéder à l'application. Cela permettrait notamment de connecter des smartphones à notre application. Nous allons essayer de créer cette fonctionnalité mais nous redoutons les sécurités qui sont implémentées dans les navigateurs et qui risquent de bloquer les connexions.

Enfin, s'il nous reste du temps, nous allons essayer de mettre en place un moyen de sortir de la zone de danger, en mettant en place un calcul d'itinéraire.

Limites, Discussion et Piste d'amélioration

- Limites actuelle de l'application

Notre application n'étant pas sans faille, nous avons souhaité créer cette partie pour discuter de ces éventuels problèmes. Tout d'abord, cette application dépend d'entreprises tierces, ce qui signifie que si ces entreprises ne transmettent pas de données, l'application ne peut pas les afficher. C'est ce qui est arrivé avec Atmo entre les fêtes, le site internet ne fonctionnait pas et aucune donnée n'était envoyée, conséquence la vue relative à la qualité de l'air était vide.

De plus, la présence de nombreuses couches régulièrement mises à jour empêche la mise en cache de ces dernières, qui pourrait nous faire économiser du temps de calcul.

Ensuite, pour créer le tunnel entre nos deux ordinateurs nous avons dû modifier le paramétrage de nos pare-feu afin d'autoriser la connexion d'un autre appareil au port local. Une cyberattaque contre Tailscale ou une erreur de notre part pourraient compromettre la sécurité de nos appareils. Mais cette connexion nous semblait essentielle afin que nous puissions travailler en binôme sur l'application.

- Limites dans le cadre d'un déploiement de l'application

La volonté de créer une telle application dans la vraie vie poserait bien plus de problèmes que pour un simple projet étudiant. Nous établissons cette liste que nous allons détailler :

- Un problème juridique
- Un problème de fiabilité technique
- Un problème d'accès
- Un problème de fiabilité de la donnée

Tout d'abord, nous pensons que cette application si elle est déployée se heurterait à des soucis juridiques. En effet, les notions de danger appellent des notions de potentielle blessure, voire de mort. Qui tenir pour responsable dans le cas où un danger ne serait pas référencé dans l'application ? Cette question empêche, selon nous, le déploiement de l'application.

La fiabilité technique est également une thématique centrale quand un outil peut éviter la mise en danger d'individus. Aujourd'hui, avec notre architecture nous ne sommes pas capables de l'assurer.

Un outil qui peut être crucial dans certaines situations se doit d'être accessible. Notre application est aujourd'hui développée sur des ordinateurs portables qui ne sont pas connectés à internet en permanence. Au-delà de cet aspect, et comme déjà dit précédemment notre outil dépend des entreprises tierces pour fonctionner. Cela signifie que si ces entreprises subissent une panne ou une attaque informatique, les informations ne seront pas disponibles sur notre application. Être dépendant est une faiblesse qui multiplie le risque de panne.

La donnée géographique en elle-même possède des failles, que nous devons connaître absolument et que nous devons aussi faire connaître à nos utilisateurs. Ces failles peuvent être intrinsèquement liées à la donnée ou le fruit de choix, que nous avons faits pour créer cette application. Par exemple, nous avons choisi, pour les inondations, de ne traiter que les risques de forte probabilité, cela signifie que nous choisissons de laisser de côté les risques de moyenne et de faible probabilité. Il serait alors erroné de penser ou de laisser croire à nos utilisateurs qu'ils seraient en parfaite sécurité en se trouvant hors de cette zone de forte probabilité.

Conclusion

Nous pensons avoir beaucoup appris en créant cette application. En plus des notions purement techniques du webmapping, nous avons également pu découvrir une thématique beaucoup plus météorologique que ce que nous avons fait jusqu'à présent. En cela, ce sujet était intéressant pour nous. Au-delà de cela, nous pouvons dire que nous avons pris du plaisir à créer cette application. De plus, le choix libre des technologies nous a permis de découvrir des logiciels un peu éloignés de la thématique purement géomatique mais qui pourront nous être utiles à l'avenir.

Toutefois, à l'heure où nous écrivons ces lignes, l'application n'est pas encore finalisée. Nous continuerons, sur le temps qu'il nous reste, à l'améliorer et à créer de nouvelles fonctionnalités.

Bibliographie :

Obligations Légales de Débroussaillage. (s. d.). Géorisques. Consulté 20 janvier 2026, à l'adresse

<https://www.georisques.gouv.fr/donnees/bases-de-donnees/obligations-legales-de-debroussaillage>

METEO FRANCE – L'API AROME. (s. d.). Consulté 20 janvier 2026, à l'adresse

<https://portail-api.meteofrance.fr/web/fr/api/AROME>

METEO FRANCE –API VIGILANCE. (s. d.). Consulté 20 janvier 2026, à l'adresse

<https://portail-api.meteofrance.fr/web/fr/liste-api/categorie/Vigilance>

API d'Atmo Data, l'agrégateur des flux open data des AASQA. (s. d.). Consulté 20 janvier 2026, à l'adresse <https://admindata.atmo-france.org/api/doc/v2>

Zonages Inondation—Rapportage 2020. (s. d.). Géorisques. Consulté 20 janvier 2026, à l'adresse

<https://www.georisques.gouv.fr/donnees/bases-de-donnees/zonages-inondation-rapportage-2020>

Annexes

Annexe 1: Méthode de mise à jour de la vigilance Météo France

1.1 Le script Python

```
import os
import datetime
import requests # Librairie standard pour télécharger
import subprocess # Pour lancer les commandes système

token = "Clé d'API très longue"
dossier_temp = r"C:\Temp_SIG" #Dossier temporaire
fichier_vigilance = os.path.join(dossier_temp, "vigilance.json")
pathQGIS= r"C:\Program Files\QGIS 3.34.11\bin"
pathogr = pathQGIS + r"\ogr2ogr.exe"
path_psql = r"C:\Program Files\PostgreSQL\18\bin\psql.exe"

base_url="https://public-api.meteofrance.fr/public/DPVigilance/v1/cartevigilance/encours"

headers = {"apikey": token}

response = requests.get(base_url, headers=headers, verify=False)
if response.status_code == 200:
    with open(fichier_vigilance, 'wb') as f:
        f.write(response.content)
    print("Fichier téléchargé !")
else:
    print(f"Erreur téléchargement : {response.text}")
    exit()

# Le fichier SQL qui fait le tri
fichier_sql=r"C:\Users\maxim\AppData\Roaming\DBeaverData\workspace6\General\Scripts\traitement_vigilance.sql"

# --- 2. AUTHENTIFICATION ---
# Configuration de la DB
db_conf = {
    "host": "localhost",
    "port": "5432",
    "dbname": "Webmapping",
    "user": "postgres",
    "password": "webmapping"
}
# On prépare l'environnement pour PSQL (le mot de passe)
env_psql = {**os.environ, "PGPASSWORD": db_conf["password"]}
base_cmd = [
    path_psql,
```

```

        "-h", db_conf["host"],
        "-p", db_conf["port"],
        "-d", db_conf["dbname"],
        "-U", db_conf["user"],
        "-q"
    ]

# Etape 2 : Import Brut
print("[2/3] Import du JSON brut...")

# 2.1 DROP et CREATE
sql_create = "DROP TABLE IF EXISTS raw_vigilance; CREATE TABLE raw_vigilance
(data jsonb);"
subprocess.run(base_cmd + ["-c", sql_create], env=env_psql)

# 2.2 COPY
sql_copy = fr"\copy raw_vigilance FROM '{fichier_vigilance}' CSV QUOTE
E'\x01' DELIMITER E'\x02';"
subprocess.run(base_cmd + ["-c", sql_copy], env=env_psql)

# Etape 3 : Traitement SQL
print("[3/3] Exécution du script SQL de traitement...")
subprocess.run(base_cmd + ["-f", fichier_sql], env=env_psql)

print("\n--- SUCCESS ! VIGILANCE METEO À JOUR. ---")

```

1.2 Le script batch de lancement

```

SET PYTHON_EXE="C:\Program Files\QGIS
3.34.11\apps\Python312\python.exe"

SET SCRIPT_PY=
"C:\Users\maxim\Desktop\Géomatique\master_2\Webmapping\Projet_dang
er\vigilance\vigilance.py"

%PYTHON_EXE% %SCRIPT_PY%

```

Annexe 2 : Méthode de mise à jour de la pluie

```

import os
import datetime
import requests # Librairie standard pour télécharger
import subprocess # Pour lancer les commandes système

# --- 1. CONFIGURATION ---
token = "Clé d'API trop longue"
dossier_temp = r"C:\Temp_SIG"
fichier_grib = os.path.join(dossier_temp, "pluie.grib2") #Traitement d'un

```



```

fichier .grib
fichier_tif = os.path.join(dossier_temp, "pluie.tif")

# Chemins des outils

path_gdal = r"C:\Program Files\QGIS 3.34.11\bin\gdal_translate.exe"
path_raster2pgsql = r"C:\Program Files\PostgreSQL\18\bin\raster2pgsql.exe"
path_psql = r"C:\Program Files\PostgreSQL\18\bin\psql.exe"

# --- 2. CALCUL DE LA DATE ---
# On prend l'heure actuelle UTC, moins 4h
now = datetime.datetime.utcnow() - datetime.timedelta(hours=4)

# On arrondit à 3h (1e Run)
hour_run = 3 * (now.hour // 3)
date_run = now.replace(hour=hour_run, minute=0, second=0, microsecond=0)

# On calcule la cible (Run + 1h)
date_target = date_run + datetime.timedelta(hours=6)

# Formatage des chaines pour l'URL
str_id = date_run.strftime("%Y-%m-%dT%H:%M:%SZ")
str_target = date_target.strftime("%Y-%m-%dT%H:%M:%SZ")

print(f"Run visé : {str_id}")
print(f"Cible visée : {str_target}")

# --- 3. TELECHARGEMENT ---
base_url =
"https://public-api.meteofrance.fr/public/arome/1.0/wcs/MF-NWP-HIGHRES-AROME
-0025-FRANCE-WCS/GetCoverage"
params = {
    "SERVICE": "WCS",
    "VERSION": "2.0.1",
    "REQUEST": "GetCoverage",
    "format": "application/wmo-grib",
    "coverageId":
f"TOTAL_PRECIPITATION__GROUND_OR_WATER_SURFACE___{str_id}_PT1H",
    "subset": [
        "lat(41,52)",
        "long(-6,10)",
        f"time({str_target})"
    ]
}
headers = {"apikey": token}

print("Téléchargement en cours...")
response = requests.get(base_url, params=params, headers=headers,

```

```

verify=False)

if response.status_code == 200:
    with open(fichier_grib, 'wb') as f:
        f.write(response.content)
    print("Fichier téléchargé !")
else:
    print(f"Erreur téléchargement : {response.text}")
    exit()

# --- 4. TRAITEMENT GDAL ---

path_qgis_root = r"C:\Program Files\QGIS 3.34.11"

# 1. GDAL_DATA (Pour les formats de fichiers)
os.environ['GDAL_DATA'] = os.path.join(path_qgis_root,
r"apps\gdal\share\gdal")

# 2. PROJ_LIB
# On pointe vers le dossier PROJ de QGIS
os.environ['PROJ_LIB'] = os.path.join(path_qgis_root, r"share\proj")

print("Conversion GRIB -> TIFF...")
cmd_gdal = [path_gdal, "-of", "GTiff", "-a_srs", "EPSG:4326", fichier_grib,
fichier_tif]
subprocess.run(cmd_gdal, stdout=subprocess.DEVNULL)

# --- 5. IMPORT POSTGIS ---
print("Importation en base...")

# Commande raster2pgsql
cmd_import = [
    path_raster2pgsql,
    "-s", "4326",
    "-d", # Drop (écraser)
    "-C", "-I", "-M",
    fichier_tif,
    "public.meteo_pluie_live"
]

process_raster = subprocess.Popen(cmd_import, stdout=subprocess.PIPE)
process_psql = subprocess.Popen(
    [path_psql, "-h", "localhost", "-d", "Webmapping", "-U", "postgres"],
    stdin=process_raster.stdout,
    env={**os.environ, "PGPASSWORD": "webmapping"} # On passe le mot de
passe ici

```

```
)
process_raster.stdout.close()
process_psql.communicate()

print("Terminé ! Données à jour.")
```

Annexe 3 : Méthode de mise à jour de la qualité de l'air

```
import os
import datetime
import requests # Librairie standard pour télécharger
import subprocess # Pour lancer les commandes système

# --- 1. CONFIGURATION ---
dossier_temp = r"C:\Temp_SIG"
fichier_atmo = os.path.join(dossier_temp, "atmo_jour.json")
pathQGIS= r"C:\Program Files\QGIS 3.34.11\bin"
pathogr = pathQGIS + r"\ogr2ogr.exe"
path_psql = r"C:\Program Files\PostgreSQL\18\bin\psql.exe"

now = datetime.datetime.utcnow()
str_now = now.strftime("%Y-%m-%d")

base_url =
f'https://data.atmo-france.org/geoserver/ind/ows?service=WFS&version=1.0.0
&request=GetFeature&typeName=ind:ind_atmo&outputFormat=application/json&CQ
L_FILTER=date_ech>={str_now}'

response = requests.get(base_url, verify=False) # verify=False evite les
erreurs SSL

w = open(fichier_atmo, 'wb')
w.write(response.content)
w.close()
# --- 5. IMPORT POSTGIS ---
print("Importation en base...")

# Commande ogr2ogr
cmd_vider = [
    path_psql,
    "-h", "localhost", "-d", "Webmapping", "-U", "postgres",
    "-c", "TRUNCATE TABLE public.france_atmo_live;"
]

subprocess.run(cmd_vider, env={**os.environ, "PGPASSWORD": "webmapping"})

cmd_import = [pathogr, "-f", "PostgreSQL", "PG:host=localhost
dbname=Webmapping user=postgres"]
```

```
password=webmapping",fichier_atmo,"-update","-append","-nln","france_atmo_  
live"]  
process_psql = subprocess.run(cmd_import,capture_output=True, text=True)  
  
print("Terminé ! Données à jour.")
```